

CSC2161 C++ PROGRAMMING FUNCTIONS

Dieudonne U



AGENDA

Topic two

Topic three

Topic four

What are functions of C++?

A function is a block of code that performs some operation. A function can optionally define input parameters that enable callers to pass arguments into the function. A function can optionally return a value as output.



BENEFITS

- MODULARITY (DECMPOSITION)
- READABILITY
- MAINTANABILITY
- REUSABILITY



FUNCTION DEFINITION

C++ HAS ONE DEFINITION RULE (ODR): A FUNCTION CAN DECLARED OR INVOKED IN MANY DIFFERENT PLACES, BUT IT IS DEFINED IN ONE PLACE.

```
#include <iostream>
bool isPrime(int);
int main()
{
 int n;
 std::cout << "Enter a positive
number\n";
 std::cin >> n;
for (int i = 2; i < n; ++i)
{
  if (isPrime(i))
    std::cout << i << "\n";
}
return 0;
}
```

```
bool isPrime(int number)
{
  if (number <= 1)
       return false;
  if (number == 2)
       return true;
 for (int i = 2; i < number; +
+i)
    if (number % i == 0)
          return false;
 return true;
}
```

C++ HAS ONE DEFINITION RULE (ODR): A FUNCTION CAN DECLARED OR INVOKED IN MANY DIFFERENT PLACES, BUT IT IS DEFINED IN ONE PLACE.

```
#include <iostream>
bool isPrime(int);
int main()
{
 int n;
 std::cout << "Enter a positive
number\n";
 std::cin >> n;
for (int i = 2; i
{
  if (isPrime(i))
    std::cout << i << "\n";
}
return 0;
}
```

PROTOTYPE

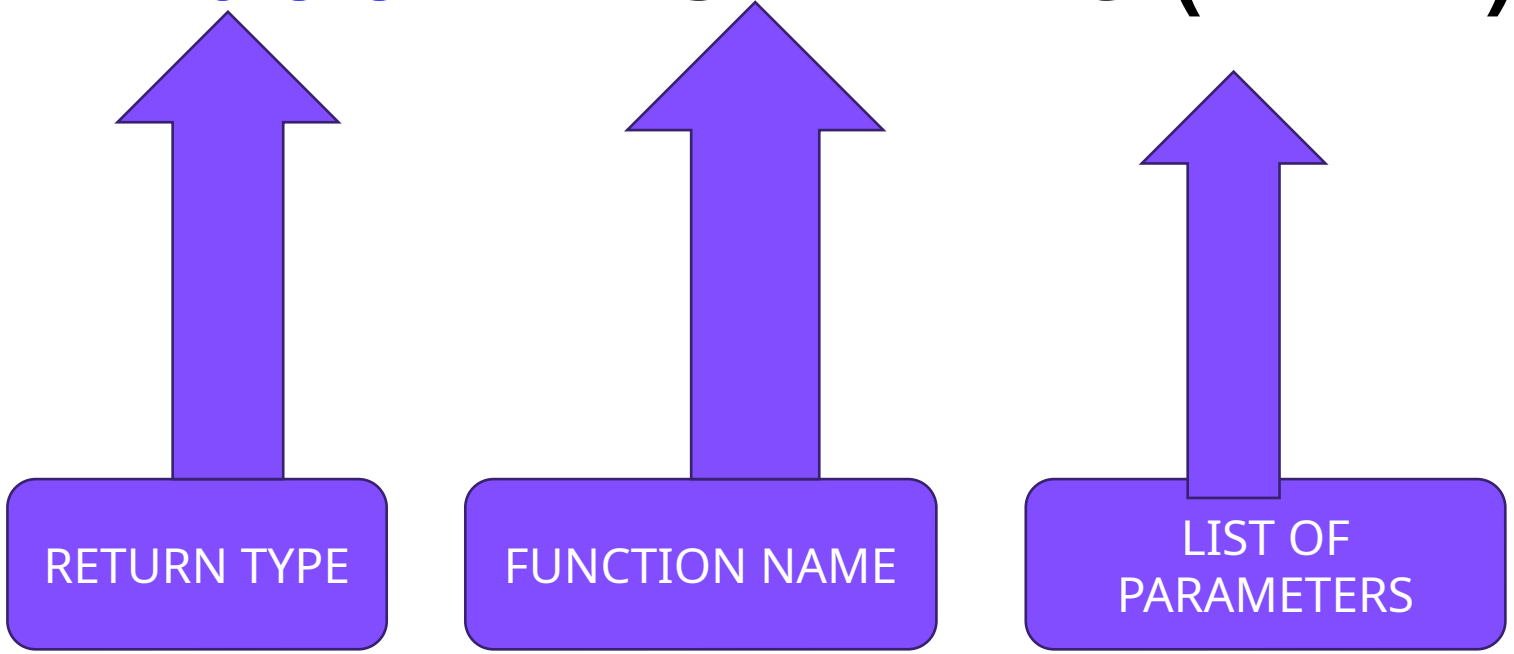
FUNCTION CALL

```
bool isPrime(int number)
{
  if (number <= 1)
       return false;
  if (number == 2)
       return true;
 for (int i = 2; i < number; +
+i)
    if (number % i == 0)
          return false;
 return true;
}
```

FUNCTION DEFINITION

FUNCTION PROTOTYPE : DECLARATION

```
bool isPrime(int);
```



Prototype is required when the function call comes before the compiler sees the function definition.

FUNCTION DEFINITION

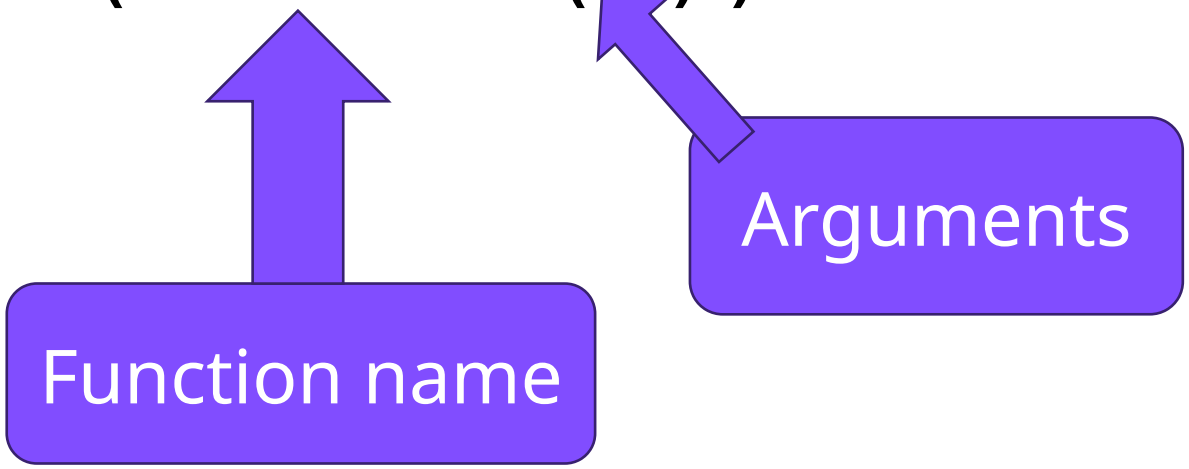
```
bool isPrime(int number)
{
    if (number <= 1)
        return false;
    if (number == 2)
        return true;
    for (int i = 2; i < number; ++i)
        if (number % i == 0)
            return false;
    return true;
}
```

FUNCTION HEADER

FUNCTION BODY

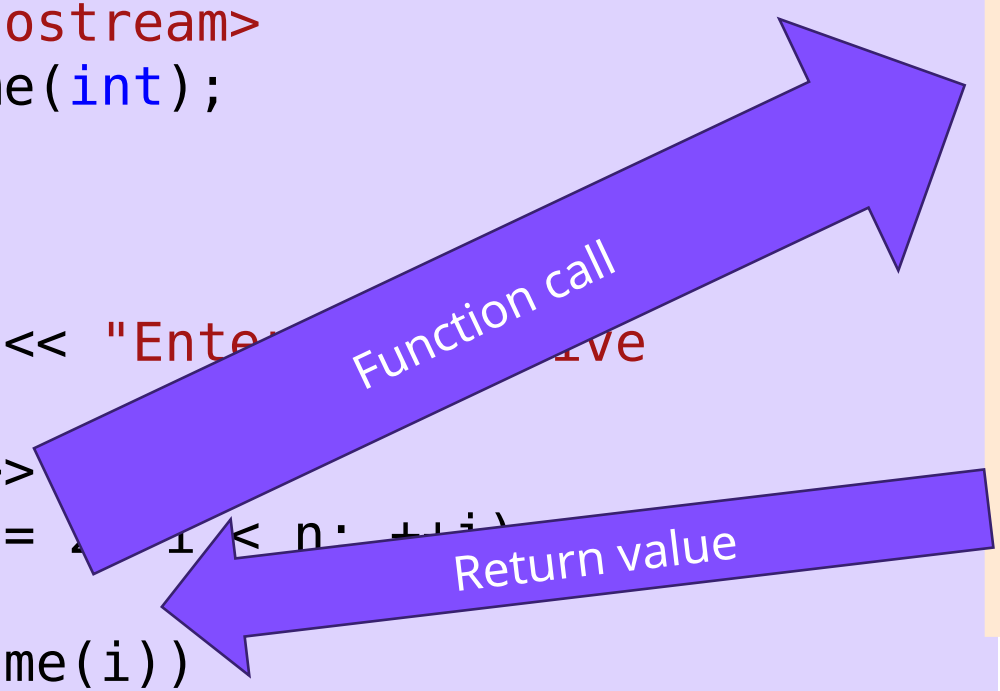
FUNCTION CALL

```
if (isPrime(i))
```



C++ HAS ONE DEFINITION RULE (ODR): A FUNCTION CAN DECLARED OR INVOKED IN MANY DIFFERENT PLACES, BUT IT IS DEFINED IN ONE PLACE.

```
#include <iostream>
bool isPrime(int);
int main()
{
 int n;
 std::cout << "Ente                ive
number\n";
 std::cin >>
for (int i =      i < n; ++i)
{
  if (isPrime(i))
    std::cout << i << "\n";
}
return 0;
}
```




```
bool isPrime(int number)
{
  if (number <= 1)
       return false;
  if (number == 2)
       return true;
 for (int i = 2; i < number; +
+i)
    if (number % i == 0)
          return false;
 return true;
}
```

Inline function

Function calls involve performance overheads. If the function is simple enough, it might be advantageous to just lay the code of the function where the function is called. That is what happens when a function is inline.

```
inline bool isEven(int n)
{
    return n % 2 == 0;
}
```



Making a function inline is a request to the compiler, it decides to implement it or not.

Compilers are smart enough to make simple functions inline, without the request of the programmer.

C++ HAS TWO MECHANISMS OF PASSING PARAMETERS TO FUNCTIONS

```
void SwapByValue(int a, int b)
{
   int temp = a;
   a = b;
   b = temp;
}
```

```
void SwapByRef(int& a, int& b)
{
  int temp = a;
  a = b;
  b = temp;
}
```

```
int main()
{
int a = 7, b = 9;
SwapByValue(a, b);
std::cout << "a= " << a << "\n";
std::cout << "b= " << b << "\n";
SwapByRef(a, b);
std::cout << "a= " << a << "\n";
std::cout << "b= " << b << "\n";
    return 0;
}
```

C++ HAS TWO MECHANISMS OF PASSING PARAMETERS TO FUNCTIONS

```
void SwapByValue(int a, int b)
{
   int temp = a;
   a = b;
   b = temp;
}
```

```
void SwapByRef(int& a, int& b)
{
  int temp = a;
  a = b;
  b = temp;
}
```

```
int main()
{
int a = 7, b = 9;
SwapByValue(a, b);
std::cout << "a= " << a << "\n";
std::cout << "b= " << b << "\n";
SwapByRef(a, b);
std::cout << "a= " << a << "\n";
std::cout << "b= " << b << "\n";
    return 0;
}
```

a= 7
b= 9

a= 9
b= 7

RECURSIVE FUNCTIONS

Recursion is the technique of making a function call itself. This technique provides a way to break complicated problems down into simple problems which are easier to solve.
Recursion may be a bit difficult to understand. The best way to figure out how it works is to experiment with it.

Example

Write the output of this program

```
int sum(int k) {
  if (k > 0) {
    return k + sum(k - 1);
  } else {
    return 0;
  }
}

int main() {
  int result = sum(10);
  cout << result;
  return 0;
}
```

Exercises

1. Write a function to calculate the factorial of a number
2. Write a function for Fibonacci series
3. Write a function to find the sum of elelents in an Array









THANK YOU

Presenter name

Email address

Website